Agent Harness

面试 100 题：题目驱动式深度精讲

Agent Engineer / Agent Systems / Harness / AI Infrastructure

Know-Why × Know-How × Failure Modes × Trade-offs × Evals

2026 深度版

基于公开工程实践、论文与所附 Agent / RAG 资料整理

编写说明与“真题”口径

重要说明 本书借鉴《剑指 Offer》“以题带知识、逐层追问、从朴素答案走向工程最优解”的教学组织方式，但不复制其原文、题解或版式。本文所有表述与结构均为重新创作。

“100 道真题”采用工程面试的严格口径：公开可验证的岗位职责、公开 Agent 系统实践、论文暴露的真实 failure mode 与生产问题，被重构为高拟真面试题；对于无法公开验证的公司内部原题，不做“某厂原题”式虚构标注。

本书的目标不是训练框架 API 记忆，而是训练一种系统性回答方式：先解释为什么（Know-Why），再说明怎么做（Know-How），随后主动讨论失败模式、权衡、评测和可观测性。Senior/Staff 面试最看重的正是这一层。

资料基线

- 所附《AI Agents in Action》：actions/tools、memory/knowledge、reasoning/evaluation、planning/feedback 的完整 Agent 组件视角。
- 所附《AI Agent 开发与应用：基于大模型的智能体构建》：上下文管理、短/长期记忆、API、向量数据库、ReAct、多智能体与业务开发链路。
- 所附《字节跳动 RAG 实践手册》：检索触发与查询理解、质量-效率-成本三维评估、全链路监控、成本归因、故障复盘、RAG 与 Agent 协同。
- 公开工程资料：Anthropic 2024-2026 Agent/Context/Eval/Sandbox/Long-running Harness 系列文章；OpenAI 2026 Agent Systems/Codex/API Agents 岗位职责与 Agents SDK；SWE-agent ACI 论文。

如何使用本书

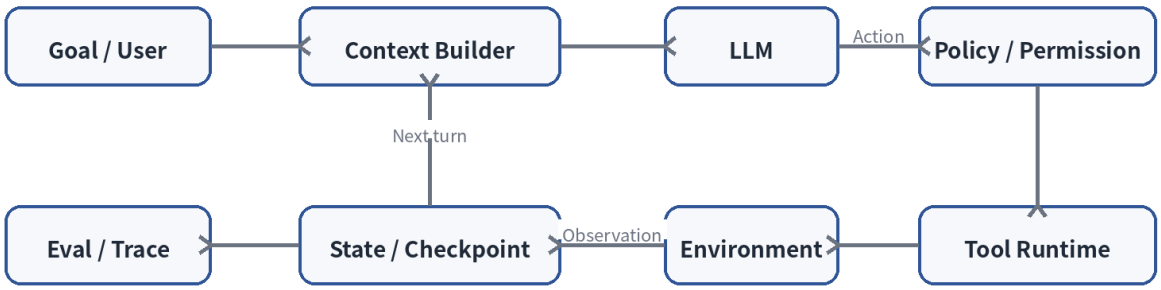
建议不要按“背答案”阅读。每题先用 90 秒口头回答，再看“核心结论”和“Know-Why”，随后尝试把 How 落到数据结构、状态机、API 或指标上。最后用“面试追问”检查自己是否只是懂概念，还是能设计生产系统。

层级	典型能力	面试信号	本书重点
L1 应用开发	Framework/API 调用	会搭 Demo	Q1-30
L2 Agent Engineer	Loop / Tools / RAG / Memory	会实现 Agent	Q1-40
L3 Harness Engineer	Checkpoint / Permission / Eval / Trace	会做可靠系统	Q41-80
L4 AI Systems	Distributed / Cost / Ablation / Runtime	会从证据优化系统	Q71-100
L5 Staff/Research Engineer	Model × Harness co-design	会定义新 runtime 原语	重点题 + 系统设计

统一答题框架：12 个问题

1. Goal：系统究竟要完成什么，什么叫成功?
2. Environment：Agent 操作的真实世界是什么?
3. Observation：模型能看见什么，哪些信息不可信?
4. Action Space：可以执行哪些工具和副作用?
5. State：状态存在哪里，哪些必须持久化?
6. Control Loop：下一步由谁决定，何时停止?
7. Failure：典型失败模式是什么?
8. Recovery：失败、超时、Crash 后如何恢复?
9. Security：权限、凭证、Sandbox、Blast Radius 如何控制?
10. Evaluation：如何证明完成，不依赖模型自述?
11. Economics：Latency / Token / Cost 如何约束?
12. Observability：如何从 Trace 还原一次失败?

图 1 Agent Harness：模型之外的控制系统



核心心智模型：LLM 是概率策略，Harness 是控制系统与运行环境。

目录

第一章 Agent Loop 与 Harness 基础

Q01-Q10 · 理解“模型之外的系统”：控制循环、状态、终止条件、Model × Harness 共设计。

第二章 Tool / ACI / MCP 与执行接口

Q11-Q20 · 工具不是附属 API，而是 Agent 的 action space。接口设计会直接改变模型行为分布。

第三章 Context Engineering 与 Memory

Q21-Q30 · 把上下文视为有限预算：选择、压缩、外置状态、跨 Session 恢复。

第四章 Planning / Orchestration / Multi-Agent

Q31-Q40 · 先判断是否需要 Agent，再判断是否需要多 Agent；复杂编排必须换来可测量收益。

第五章 Long-Horizon / Durable Agent

Q41-Q50 · 长任务不是无限循环，而是可分解、可验证、可交接、可恢复的 durable workflow。

第六章 Permission / Sandbox / Security

Q51-Q60 · 安全的核心是缩小 blast radius，并让授权、身份、凭证和执行边界成为系统原语。

第七章 Evaluation / Observability

Q61-Q70 · Agent 评测必须面向 trajectory + outcome；生产 trace 要进入 eval 和 regression 闭环。

第八章 Reliability / Distributed Agent Systems

Q71-Q80 · Agent Harness 本质上会遭遇分布式系统问题：重试、幂等、取消、并发、一致性和故障恢复。

第九章 Cost / Latency / Performance

Q81-Q90 · 优化目标不是“最强模型”，而是 Quality × Reliability × Latency × Cost 的 Pareto 前沿。

第十章 System Design / Coding / Debugging

Q91-Q100 · 把前九章组合成完整系统设计：Coding Agent、Research Agent、平台、多租户、Trace、Checkpoint 与回归定位。

附录 A：30 天冲刺计划　附录 B：Agent System Design 答题模板　附录 C：Failure Taxonomy　附录 D：参考资料与延伸阅读

为什么 2026 年要单独准备 Agent Harness

行业变化 公开岗位已经把 Agent Engineer 的边界从“Prompt + RAG + Function Calling”扩展到 execution loop、context/memory、sandboxing、permissions、orchestration、long-running workflow、observability、evals、latency/cost 与分布式可靠性。

OpenAI 公开的 Codex Core Agents / API Agents / Cloud Agents 岗位，把 agent harness、execution loop、sandboxed execution、stateful workflows、memory、delegation、secure environment connectivity、identity/permissions、observability、evaluations、tokens/latency/cost 等列为核心职责。这意味着面试已经从“LLM 应用开发”进入“AI Systems Engineering”。[W1][W2][W3]

Anthropic 的工程实践进一步说明：长任务需要 initializer + incremental progress + structured handoff；context 需要 compaction、note-taking 或 multi-agent；Agent eval 要以 task/trial/trace/outcome/grader 建模；sandbox 必须同时考虑 filesystem 与 network isolation。[W4][W5][W6][W7]

SWE-agent 则从研究侧证明了 ACI（Agent-Computer Interface）的设计本身会显著改变 Agent 的行为和任务成功率。因此，“Harness 是能力放大器”不是产品口号，而是可以通过实验观察和消融验证的系统事实。[W8]

本书的三根主轴

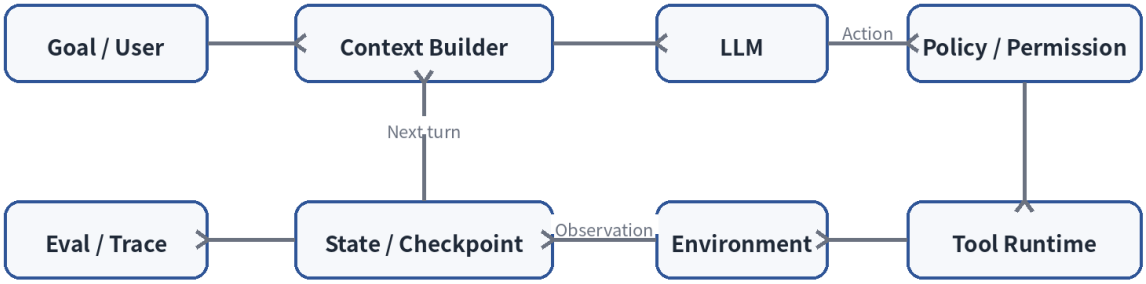
Intelligence Amplification	Reliability Engineering	Capability Containment
用 context/tools/planning/environment 放大模型能力	用 state/checkpoint/idempotency/eval 把概率行为工程化	用 permission/sandbox/identity/audit 缩小 blast radius

第一章 Agent Loop 与 Harness 基础

本章核心 理解“模型之外的系统”：控制循环、状态、终止条件、Model × Harness 共设计。

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

图 1 Agent Harness：模型之外的控制系统



本章面试目标

- 能自己写 Agent Loop，而不是只会框架。
- 能区分 Model failure 与 Harness failure。
- 能解释 stop condition、state machine 与 environment verification。

Q01 什么是 Agent Harness？它和 Agent Framework 有什么区别？

难度：★★★☆☆　　高频：★★★★★　　岗位：Agent / Applied AI

一句话结论 Harness 定义观察、行动、状态、控制循环和安全边界；Framework 只是实现这些能力的工程抽象。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Harness 定义观察、行动、状态、控制循环和安全边界；Framework 只是实现这些能力的工程抽象。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 先用最小循环说明 Harness，再补 Context、Tool、State、Permission、Trace、Eval 等横切能力。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把 Harness 等同 Prompt
- 只报 LangChain/LangGraph 名称
- 忽略环境和终止语义

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q02 请从零写出一个最小 Agent Loop。

难度：★★★☆☆ 高频：★★★★★ 岗位：Agent / Applied AI

一句话结论 最小循环是 Model -> Tool -> Observation -> Model，生产化的难点在于约束循环而不是写出循环。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：最小循环是 Model -> Tool -> Observation -> Model，生产化的难点在于约束循环而不是写出循环。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 实现 stop condition、max steps、tool result 回填
- 再扩展 timeout、retry、checkpoint、budget

```
messages = [user_input]
for step in range(MAX_STEPS):
    ctx = build_context(messages, state)
    resp = model(ctx, tools=visible_tools(state))
    trace(resp)
    if resp.final and verify_goal(state, resp):
        return resp.final
    for call in resp.tool_calls:
        decision = policy.evaluate(call, state)
        result = tool_runtime.execute(call, decision)
        messages.append(tool_result(call.id, result))
    checkpoint(state)
```

Failure Modes：常见低分回答

- 只写 while True

- 未绑定 tool_call_id
- 没有 max turns 与异常边界

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q03 Agent Loop 有哪些 Stop Condition?

难度：★★★☆☆ 高频：★★★★★ 岗位：Agent / Applied AI

一句话结论 必须同时具备模型声明型终止和环境验证型终止，并受时间、成本、步骤和策略预算约束。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：必须同时具备模型声明型终止和环境验证型终止，并受时间、成本、步骤和策略预算约束。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- final answer、goal state、structured completion、max turn/token/cost、fatal error、human interrupt。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只信模型说“完成”
- 没有硬预算
- 测试失败仍结束

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q04 为什么 ReAct 不是一个完整的 Agent 架构?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Applied AI

一句话结论 ReAct 是认知循环模式，不覆盖持久状态、权限、安全、恢复、评测和运维。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：ReAct 是认知循环模式，不覆盖持久状态、权限、安全、恢复、评测和运维。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 先解释 Reason-Act-Observe，再列出 production runtime 需要的外层系统。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把 ReAct 与生产 runtime 混为一谈

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q05 Agent 和普通 Workflow 的本质区别是什么?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Applied AI

一句话结论 Workflow 的路径主要由程序决定；Agent 的下一步路径由模型在环境反馈下动态选择。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Workflow 的路径主要由程序决定；Agent 的下一步路径由模型在环境反馈下动态选择。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 用动态 action selection、长尾复杂度和不确定性说明何时值得 Agent 化。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 凡是用了 LLM 就叫 Agent
- 固定 DAG 也强行上自治循环

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q06 什么问题根本不应该使用 Agent?

难度：★★★☆☆　　高频：★★★☆☆　　岗位：Agent / Applied AI

> **一句话结论** 确定性强、流程固定、高风险且不可验证、极端低延迟的任务通常不应 Agent 化。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：确定性强、流程固定、高风险且不可验证、极端低延迟的任务通常不应 Agent 化。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 用 ambiguity、dynamic tool selection、verification ability、latency budget 做决策。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 为了“智能”牺牲可靠性
- 没有非 Agent baseline

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q07 Model 能力和 Harness 能力如何区分?

难度：★★★★☆ 高频：★★★★★ 岗位：Agent / Applied AI

> **一句话结论** 失败可能来自模型、上下文、工具接口、执行环境、权限、运行时或评测，必须做系统级归因。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：失败可能来自模型、上下文、工具接口、执行环境、权限、运行时或评测，必须做系统级归因。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 建立 failure taxonomy，做 old/new model × old/new harness 的交叉 ablation。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 看到失败就改 prompt 或换模型

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q08 为什么同一个模型换一个 Harness，Benchmark 可能差很多?

难度：★★★★☆ 高频：★★★★☆ 岗位：Agent / Applied AI

> **一句话结论** Harness 改变 observation space、action space、feedback quality 和 search strategy，因此会改变模型策略分布。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Harness 改变 observation space、action space、feedback quality 和 search strategy，因此会改变模型策略分布。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 对 tool schema、context、error feedback、edit interface 做独立消融。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence SWE-agent 论文把 ACI 作为核心研究变量，显示“给模型什么接口”会显著改变软件工程任务表现。[W8]

Failure Modes：常见低分回答

- 把 Benchmark 分数只归功于模型参数

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q09 Agent Runtime 最重要的抽象是什么?

难度：★★★☆☆　　高频：★★★☆☆　　岗位：Agent / Applied AI

一句话结论 把 Agent 建模为 event-driven state machine + probabilistic policy，比“聊天机器人”抽象更可靠。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：把 Agent 建模为 event-driven state machine + probabilistic policy，比“聊天机器人”抽象更可靠。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 定义 Run/Turn/Event/State/Action/Observation，并让每个状态变化可追踪、可恢复。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 消息数组既当状态又当日志

- 无法区分业务状态与模型上下文

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q10 如果不用 LangChain/LangGraph，你如何自己实现 Agent?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Applied AI

> **一句话结论** 最小需要 Model Adapter、Tool Registry、Agent Loop、State Store、Runtime Policy；其余能力逐层加。

面试官在考什么

你是否理解 Agent 的“控制平面”，以及为什么生产系统不能把所有责任推给模型。

Know-Why：为什么

真正的 Agent 工程不是“让模型多想几步”，而是把一个概率策略放进可约束、可观测、可恢复的软件运行时。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：最小需要 Model Adapter、Tool Registry、Agent Loop、State Store、Runtime Policy；其余能力逐层加。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 先 200 行实现，再增加 checkpoint、permissions、tracing、eval、compaction、queue。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 一开始造完整框架
- 或反过来完全依赖框架黑盒

Trade-off：主动说出代价

更强的控制和可观测性会增加 runtime 复杂度；应从最小循环开始，只为真实 failure mode 增加机制。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第一章复盘：把 10 道题串成一个系统

本章一句话 从 Agent Loop 出发，加入 stop condition 和 state，再用 failure taxonomy 区分 Model 与 Harness，最终形成可独立于框架实现的 Runtime Kernel。

第二章 Tool / ACI / MCP 与执行接口

本章核心 工具不是附属 API，而是 Agent 的 action space。接口设计会直接改变模型行为分布。

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

本章面试目标

- 能把工具当模型接口设计，而不是普通 REST 封装。
- 理解 schema、error、idempotency 与 tool retrieval。
- 知道 API/GUI/MCP 的边界和风险。

Q11 怎样设计一个 LLM-friendly Tool?

难度： ★★★☆☆ **高频：** ★★★★★ **岗位：** Agent / Applied AI

一句话结论 好的工具语义清晰、边界明确、结果可观察、错误可恢复、输出 token-efficient。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：好的工具语义清晰、边界明确、结果可观察、错误可恢复、输出 token-efficient。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 定义单一职责、明确参数、结构化返回、错误类型、示例和负例。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 的工具设计实践强调清晰命名、描述、可组合性与 token-efficient 结果，并主张用 eval 反向优化工具本身。[W9]

Failure Modes：常见低分回答

- 万能 manage() 工具
- 字符串塞所有参数
- 返回大段日志

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q12 Tool 越多 Agent 能力是不是越强？

难度：★★★☆☆　　高频：★★★☆☆　　岗位：Agent / Applied AI

> **一句话结论** 工具数量增加会带来选择歧义、schema token、错误调用和安全面扩大，需要 tool retrieval。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：工具数量增加会带来选择歧义、schema token、错误调用和安全面扩大，需要 tool retrieval。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 按 namespace/family 路由，动态暴露 Top-K 相关工具。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 一次暴露数百工具
- 功能重叠不治理

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q13 两个 Tool 功能重叠应该怎么办？

难度：★★★☆☆　　高频：★★★★☆　　岗位：Agent / Applied AI

一句话结论 工具边界需要像优秀 API 一样可解释；功能重叠首先合并，其次明确 usage boundary。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：工具边界需要像优秀 API 一样可解释；功能重叠首先合并，其次明确 usage boundary。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 统一命名、加正反例、做 selection eval、必要时按任务阶段隐藏。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只靠 prompt 解释细微差别

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q14 Tool Schema 为什么比 Prompt 更容易被低估?

难度： ★★★☆☆ **高频：** ★★★★☆ **岗位：** Agent / Applied AI

一句话结论 Schema 是模型行动空间的类型系统；更好的结构能减少隐式解析和非法组合。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Schema 是模型行动空间的类型系统；更好的结构能减少隐式解析和非法组合。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 用 enum、required、typed objects、constraints 把业务规则前移。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 所有参数都 string
- 复杂结构让模型自己编码

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q15 Tool Error 应该怎样返回给 Agent?

难度：★★★☆☆　　高频：★★★☆☆　　岗位：Agent / Applied AI

一句话结论 错误必须告诉 Agent：发生了什么、是否可重试、是否应改参数、是否需要换工具或人工介入。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：错误必须告诉 Agent：发生了什么、是否可重试、是否应改参数、是否需要换工具或人工介入。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- error_type、retryable、retry_after、safe_message、partial_result。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只返回 500
- 把堆栈和内部 secret 全部回填模型

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?

- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q16 为什么 Tool 必须考虑 Idempotency?

难度：★★★★☆ 高频：★★★★★ 岗位：Agent / Applied AI

一句话结论 超时与重试会导致“不知道上一次是否成功”，副作用动作因此必须支持幂等和去重。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：超时与重试会导致“不知道上一次是否成功”，副作用动作因此必须支持幂等和去重。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- operation_id/idempotency_key + persisted status + dedupe + reconciliation。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
operation_id = stable_id(run_id, tool_call_id)
status = store.get_operation(operation_id)
if status == 'SUCCEEDED':
    return status.result
result = perform_side_effect()
store.mark_succeeded(operation_id, result)
return result
```

工程证据 / 研究结论

Evidence 这不是 LLM 特有问题，而是副作用调用在超时/重试下的经典分布式语义；Agent 只是让该问题出现得更频繁、更难预测。

Failure Modes：常见低分回答

- 对扣款、发信、创建订单盲目 retry

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?

- 如果工具已经产生副作用但响应丢失，系统如何确定下一步？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q17 MCP 中 Resources、Prompts、Tools 应如何理解？

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Applied AI

> **一句话结论** Resources 提供上下文，Prompts 提供可复用交互模板，Tools 提供可执行动作；安全强度应随 action capability 增长。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Resources 提供上下文，Prompts 提供可复用交互模板，Tools 提供可执行动作；安全强度应随 action capability 增长。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 把 MCP 接入 Tool Registry，同时做权限、schema 校验和 server trust 分级。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把远程 MCP 工具默认视为可信
- 把资源内容当系统指令

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q18 MCP Server 越多越好吗？

难度：★★★☆☆ 高频：★★★☆☆ 岗位：Agent / Applied AI

> **一句话结论** MCP 扩展能力的同时扩大 tool ambiguity、latency、context 和攻击面，需要发现与按需加载。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：MCP 扩展能力的同时扩大 tool ambiguity、latency、context 和攻击面，需要发现与按需加载。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- server registry、capability metadata、tool search、lazy connection、health check。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 所有 server 全连接、全 schema 常驻上下文

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q19 API Tool 和 Computer Use 如何选择?

难度：★★★☆☆　　高频：★★★★☆　　岗位：Agent / Applied AI

> **一句话结论** 优先结构化 API；只有缺少 API、需要视觉理解或遗留 UI 时才使用 computer use。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：优先结构化 API；只有缺少 API、需要视觉理解或遗留 UI 时才使用 computer use。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- API/DOM/GUI 混合，根据确定性、延迟、可验证性路由。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 能点界面就不用 API
- 把 GUI 操作当确定性事务

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q20 如果有 1000 个 Tools，怎么设计 Tool Selection?

难度：★★★★☆　　高频：★★★★☆　　岗位：Agent / Applied AI

> **一句话结论** 先检索能力，再让模型在小候选集内决策；权限过滤应尽可能在暴露前完成。

面试官在考什么

你是否能把模型调用与工具执行之间的接口设计成稳定、可恢复、可测试的 action system。

Know-Why：为什么

工具接口是给模型使用的“人机接口”。接口语义、错误结构和结果长度会直接决定模型能否稳定采取正确动作。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：先检索能力，再让模型在小候选集内决策；权限过滤应尽可能在暴露前完成。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- semantic retrieval + namespace + metadata + task classifier + policy filter。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把 1000 个 schema 一次塞进 prompt

Trade-off：主动说出代价

更细工具提高可解释性，但工具数和 round-trip 可能上升；更粗工具减少调用次数，却扩大参数组合和副作用范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第二章复盘：把 10 道题串成一个系统

本章一句话 从单个 Tool 的语义与 schema，扩展到错误恢复、幂等和 tool retrieval，最后把 MCP/API/Computer Use 统一到 action registry 与 policy 层。

第三章 Context Engineering 与 Memory

本章核心 把上下文视为有限预算：选择、压缩、外置状态、跨 Session 恢复。

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

图 3 Context Engineering：不是“塞更多”，而是选择本轮最有用的信息



本章面试目标

- 能设计 context budget 与 memory policy。
- 理解 compaction/reset/cross-session state。
- 能把 RAG 从静态检索升级为 agentic retrieval。

Q21 Prompt Engineering 和 Context Engineering 有什么区别?

难度：★★★☆☆ 高频：★★★★★ 岗位：Agent / Applied AI

一句话结论 Prompt 关注如何表达指令；Context Engineering 关注本轮究竟应该让模型看到哪些信息。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Prompt 关注如何表达指令；Context Engineering 关注本轮究竟应该让模型看到哪些信息。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 统一管理 prompt、history、memory、RAG、state、tool docs 和 recent observations。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

> **Evidence** Anthropic 将 context engineering 描述为：优化每次采样时模型看到的 token 配置，而不是只寻找一句更好的 prompt。[W6]

Failure Modes：常见低分回答

- 把所有问题都归结为 system prompt

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q22 为什么“上下文越多越好”是错误的?

难度：★★★☆☆　　高频：★★★★☆　　岗位：Agent / Applied AI

> **一句话结论** 更多 token 会引入噪声、冲突、lost-in-the-middle、成本和延迟；目标是单位 token 的决策价值。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：更多 token 会引入噪声、冲突、lost-in-the-middle、成本和延迟；目标是单位 token 的决策价值。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- relevance scoring、recency、authority、dedupe、summarize、reference artifacts。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 整段日志、整库文档、全部历史无差别注入

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q23 什么是 Context Rot / Context Pollution?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Applied AI

> **一句话结论** 长上下文会累积过时信息、错误假设和低相关内容，使模型注意力和决策质量下降。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：长上下文会累积过时信息、错误假设和低相关内容，使模型注意力和决策质量下降。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 定期 compact、reset、重检 state、清理 tool outputs，保留关键 decision record。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只等待更大 context window

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q24 Context Compaction 和 Context Reset 有什么区别?

难度：★★★★☆ 高频：★★★★★ 岗位：Agent / Applied AI

> **一句话结论** Compaction 保连续性但可能信息损失；Reset 用结构化 handoff 换 clean slate，适合长任务阶段切换。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Compaction 保连续性但可能信息损失；Reset 用结构化 handoff 换 clean slate，适合长任务阶段切换。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 根据任务阶段、summary fidelity、context anxiety 和 artifact 完整度选择。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 的长上下文实践把 compaction、structured note-taking、multi-agent 作为跨 context window 的主要手段；后续长期 Harness 又进一步使用结构化 handoff。[W4][W6]

Failure Modes：常见低分回答

- 盲目摘要
- 或每轮都 reset 导致交接开销巨大

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 哪些信息应该保留原文引用，哪些可以摘要，哪些必须外置?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q25 Short-term Memory 和 Long-term Memory 应如何设计?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Applied AI

一句话结论 短期记忆服务当前任务，长期记忆服务跨会话复用；二者写入策略、TTL 和隐私范围不同。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：短期记忆服务当前任务，长期记忆服务跨会话复用；二者写入策略、TTL 和隐私范围不同。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- working/task/semantic/episodic/user memory 分层，定义 scope、confidence、TTL。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 所有历史写 vector DB
- 把临时错误永久化

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 哪些信息应该保留原文引用，哪些可以摘要，哪些必须外置?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q26 Agent 应该什么时候写 Memory?

难度：★★★☆☆　　高频：★★★★☆　　岗位：Agent / Applied AI

一句话结论 Memory 写入是策略问题：未来价值、稳定性、置信度、作用域和隐私必须同时满足。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Memory 写入是策略问题：未来价值、稳定性、置信度、作用域和隐私必须同时满足。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- salience filter + structured extraction + dedupe + expiration + user control。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 每轮自动保存全部对话

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 哪些信息应该保留原文引用，哪些可以摘要，哪些必须外置？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q27 RAG 和 Agentic Search 有什么区别？

难度：★★★☆☆ 高频：★★★☆☆ 岗位：Agent / Applied AI

一句话结论 传统 RAG 是一次检索前置；Agentic Search 会根据证据迭代查询、验证和补充搜索。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：传统 RAG 是一次检索前置；Agentic Search 会根据证据迭代查询、验证和补充搜索。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- query planning、multi-hop retrieval、evidence verification、citation、budget stop。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把多次同 query 检索当 agentic search

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q28 Conversation History 应全部发送给模型吗？

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Applied AI

一句话结论 通常应组合 recent turns + summary + relevant history + task state，而不是完整历史。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：通常应组合 recent turns + summary + relevant history + task state，而不是完整历史。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 滑动窗口、semantic retrieval、summary、pinned facts、state snapshot。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 历史永不裁剪
- 摘要覆盖事实原文且无引用

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 哪些信息应该保留原文引用，哪些可以摘要，哪些必须外置?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q29 如何让 Agent 跨 Session 工作?

难度：★★★★☆　　高频：★★★★★　　岗位：Agent / Applied AI

> **一句话结论** 跨 Session 的关键不是聊天记录，而是 durable external state 与可验证环境。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：跨 Session 的关键不是聊天记录，而是 durable external state 与可验证环境。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- task state、progress artifact、git/checkpoint、artifact store、resume protocol。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence 所附 Agent 资料把短期/长期记忆区分为不同存储层；生产 Harness 还必须再加入 task state/checkpoint，避免把“记忆”误当成“任务真相”。[B1][B2]

Failure Modes：常见低分回答

- 把上次 summary 作为唯一状态

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 哪些信息应该保留原文引用，哪些可以摘要，哪些必须外置?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q30 Context Cache 应怎样设计?

难度：★★★☆☆ 高频：★★★☆☆ 岗位：Agent / Applied AI

一句话结论 区分 static、semi-static、dynamic context，利用 prefix/retrieval/tool cache，同时防 stale data。

面试官在考什么

你是否真正理解 context 是 runtime state 的投影，而不是聊天记录的别名。

Know-Why：为什么

上下文是有限资源。稳定 Harness 会把长期事实、任务状态和临时观察分开，并让每一轮只携带对当前决策最有价值的信息。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：区分 static、semi-static、dynamic context，利用 prefix/retrieval/tool cache，同时防 stale data。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- versioned cache key、TTL、content hash、invalidation、sensitive-data policy。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 缓存动态权限和旧环境状态

Trade-off：主动说出代价

更多上下文提升召回，但降低信噪比并增加成本；更激进压缩降低成本，却可能丢失未来才显重要的细节。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 哪些信息应该保留原文引用，哪些可以摘要，哪些必须外置?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第三章复盘：把 10 道题串成一个系统

本章一句话 从单轮 prompt 走向 context budget，再将 working memory、long-term memory、RAG 和 durable state 分层，解决跨 Session 与长上下文退化。

第四章 Planning / Orchestration / Multi-Agent

本章核心 先判断是否需要 Agent，再判断是否需要多 Agent；复杂编排必须换来可测量收益。

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

本章面试目标

- 能判断何时不用 multi-agent。
- 掌握 manager、handoff、orchestrator-worker、evaluator。
- 能讨论并发、共享状态和 fan-out 预算。

Q31 Single Agent 和 Multi-Agent 怎么选?

难度：★★★★☆ 高频：★★★★★ 岗位：Agent / Harness

一句话结论 默认单 Agent；只有并行性、上下文隔离或专业能力带来的收益超过协调成本时才多 Agent。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：默认单 Agent；只有并行性、上下文隔离或专业能力带来的收益超过协调成本时才多 Agent。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 先 single baseline，再做 fan-out ablation，看质量、成本、延迟。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 的 Building Effective Agents 明确建议从最简单可行方案开始，只在任务结构确实需要时增加 orchestrator-worker 或 evaluator-optimizer。[W9]

Failure Modes：常见低分回答

- 多 Agent 当默认架构

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如何量化 multi-agent 增益是否覆盖协调与 token 成本?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q32 Routing 和 Agent 有什么区别?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Harness

> **一句话结论** Routing 是一次或少量分类决策；Agent 是反复根据观察选择下一步动作。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Routing 是一次或少量分类决策；Agent 是反复根据观察选择下一步动作。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 如果路径可枚举，用 router/DAG
- 只有长尾动态路径才自治

Failure Modes：常见低分回答

- 把 LLM classifier 包装成 Agent 复杂化系统

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q33 什么是 Orchestrator-Worker Pattern?

难度：★★★★☆ 高频：★★★★★ 岗位：Agent / Harness

> **一句话结论** Orchestrator 动态分解任务并分配 worker，适合未知分解和可并行子问题。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Orchestrator 动态分解任务并分配 worker，适合未知分解和可并行子问题。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- decompose、dispatch、budget、aggregate、dedupe、conflict resolution。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- worker 无限 fan-out
- 没有独立性判断

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如何量化 multi-agent 增益是否覆盖协调与 token 成本?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q34 Manager-as-Tool 和 Handoff 有什么区别?

难度：★★★☆☆　　高频：★★★★☆　　岗位：Agent / Harness

> **一句话结论** Manager-as-Tool 保持主 Agent 控制；Handoff 转移当前控制权，适用场景和上下文策略不同。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Manager-as-Tool 保持主 Agent 控制；Handoff 转移当前控制权，适用场景和上下文策略不同。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 根据 final-answer ownership、guardrail ownership、specialist autonomy 选择。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把 handoff 当普通函数调用，不处理历史和权限

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q35 Agent 并行化最大的问题是什么?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Agent / Harness

> **一句话结论** 并行难点不是启动 worker，而是依赖、重复工作、共享状态冲突和聚合。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：并行难点不是启动 worker，而是依赖、重复工作、共享状态冲突和聚合。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- DAG dependency、work partition、global budget、merge protocol、barrier。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 并发修改同一文件/资源无锁

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如何量化 multi-agent 增益是否覆盖协调与 token 成本?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q36 Evaluator-Optimizer 为什么有效?

难度：★★★★☆ 高频：★★★★★ 岗位：Agent / Harness

一句话结论 生成与判断分离可以降低自我宽容，并把模糊质量要求转成明确可评分标准。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：生成与判断分离可以降低自我宽容，并把模糊质量要求转成明确可评分标准。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 先定义 rubric/contract，再 generator -> evaluator -> targeted revision。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence 2026 的 long-running application harness 实验采用 planner-generator-evaluator，并把主观质量转成可评分 criteria，说明 evaluator 的价值来自独立标准而不是“再问一次模型”。[W5]

Failure Modes：常见低分回答

- 没有明确标准就无限自我批评

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q37 Subagent 的独立 Context Window 有什么价值?

难度： ★★★★☆ **高频：** ★★★★☆ **岗位：** Agent / Harness

一句话结论 独立 context 同时提供 token scale-out 与 reasoning path diversity，但需要高质量压缩回传。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：独立 context 同时提供 token scale-out 与 reasoning path diversity，但需要高质量压缩回传。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- worker 只回传 evidence/decision/artifact refs，而不是完整 transcript。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把所有子 Agent history 合并回主 Agent

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如何量化 multi-agent 增益是否覆盖协调与 token 成本?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q38 Multi-Agent 怎么共享状态?

难度：★★★☆☆　　高频：★★★★☆　　岗位：Agent / Harness

一句话结论 关键状态必须 machine-readable、versioned、auditable，不能只依赖自然语言口头交接。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：关键状态必须 machine-readable、versioned、auditable，不能只依赖自然语言口头交接。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- shared DB、artifact store、blackboard、event bus、structured handoff。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 多个 Agent 各自维护“真相”

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?

- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q39 如何控制 Agent Fan-out?

难度：★★★★☆ 高频：★★★☆☆ 岗位：Agent / Harness

> **一句话结论** 通过 max_children、max_depth、global tokens/cost/deadline 和 expected marginal value 控制递归扩张。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：通过 max_children、max_depth、global tokens/cost/deadline 和 expected marginal value 控制递归扩张。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- semaphore + budget allocator + spawn policy + stop policy。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 让模型自由无限创建 subagent

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如何量化 multi-agent 增益是否覆盖协调与 token 成本?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q40 为什么 Multi-Agent 有时比 Single Agent 更差?

难度：★★★★☆ 高频：★★★★☆ 岗位：Agent / Harness

> **一句话结论** 协调、上下文碎片、重复探索和冲突会吃掉并行收益，尤其在强耦合任务中。

面试官在考什么

你是否能控制 orchestration complexity，并用任务结构而不是流行概念决定架构。

Know-Why：为什么

编排的价值来自任务分解与并行，而不是 Agent 数量本身。任何增加的 Agent 都必须证明它抵消了协调和 token 成本。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：协调、上下文碎片、重复探索和冲突会吃掉并行收益，尤其在强耦合任务中。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 用 task independence、communication ratio、merge difficulty 做架构判断。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只看并行速度，不看 token 和错误率

Trade-off：主动说出代价

并行和专门化可以提升 breadth，但会增加 token、协调、冲突和 handoff 损耗。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如何量化 multi-agent 增益是否覆盖协调与 token 成本?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第四章复盘：把 10 道题串成一个系统

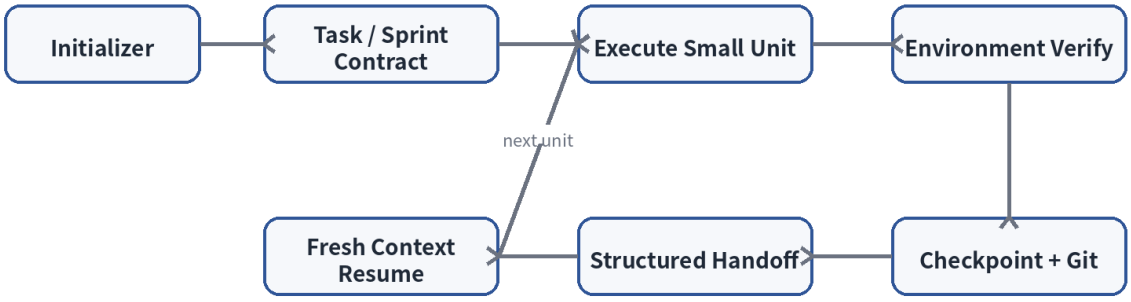
本章一句话 从 single agent baseline 开始，只在独立性与并行收益明确时引入 worker；用 manager/handoff/evaluator 等模式控制分工和质量。

第五章 Long-Horizon / Durable Agent

本章核心 长任务不是无限循环，而是可分解、可验证、可交接、可恢复的 durable workflow。

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

图 2 Long-Running Agent：把一次对话变成可恢复的任务系统



本章面试目标

- 能把小时级任务拆成 durable sprint。
- 理解 checkpoint/recovery/HITL。
- 知道“环境验证”为什么比模型自述可靠。

Q41 为什么 Long-Running Agent 会随着时间变差?

难度： ★★★★☆ **高频：** ★★★★★ **岗位：** Agent / Harness

一句话结论 context growth、plan drift、state drift、错误积累和环境变化会让行为逐步偏离目标。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：context growth、plan drift、state drift、错误积累和环境变化会让行为逐步偏离目标。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 短 sprint、验证、checkpoint、replan、fresh context、immutable goal。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 对长任务的实验指出，仅有 compaction 仍不足以稳定完成大型应用；initializer、增量进度和跨 session artifact 才能形成长期工作闭环。[W4]

Failure Modes：常见低分回答

- 把 max_steps 调大

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q42 为什么 progress file 很重要?

难度： ★★★★☆ **高频：** ★★★☆☆ **岗位：** Agent / Harness

一句话结论 结构化进度文件把隐式思维转为 durable handoff artifact，支持跨 context/session 恢复。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：结构化进度文件把隐式思维转为 durable handoff artifact，支持跨 context/session 恢复。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- done/failed/current/next/open questions/verification evidence 固定 schema。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence 长期 coding harness 使用 progress artifact 和 git history 让新的 context 快速恢复：关键知识被外置，而不是寄希望于上一轮上下文永远存在。[W4]

Failure Modes：常见低分回答

- 只写“完成了一些工作”

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q43 Initializer Agent 有什么作用?

难度：★★★★☆ 高频：★★★★☆ 岗位：Agent / Harness

一句话结论 Initializer 把高层目标转成可执行环境、任务列表、验收标准和初始状态，为后续增量工作铺路。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Initializer 把高层目标转成可执行环境、任务列表、验收标准和初始状态，为后续增量工作铺路。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- repo inspect、baseline tests、task decomposition、progress file、initial checkpoint。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 首个 coding agent 同时规划整个项目又开始大改

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q44 什么是 Sprint Contract / Definition of Done?

难度：★★★★☆ 高频：★★★★☆ 岗位：Agent / Harness

一句话结论 在实现前约定“做什么、如何验证”，把模糊目标转换为可被独立 evaluator 检查的契约。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：在实现前约定“做什么、如何验证”，把模糊目标转换为可被独立 evaluator 检查的契约。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- acceptance criteria、tests、artifact expectations、non-goals。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 实现后才临时定义成功标准

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q45 Checkpoint 应该保存什么?

难度：★★★★★ 高频：★★★★★ 岗位：Agent / Harness

一句话结论 保存无法安全重建的最小状态：run/task state、event offset、environment version、pending actions、budget。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：保存无法安全重建的最小状态：run/task state、event offset、environment version、pending actions、budget。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- checkpoint + event log
- 将大对象存 artifact store，仅保存引用

```
Checkpoint = {
  run_id, task_id, schema_version, status,
  goal_ref, plan_ref, task_state, event_offset,
  environment_version, pending_actions,
  remaining_budget, artifact_refs
}
```

工程证据 / 研究结论

Evidence Checkpoint 的工程原则是“保存不可安全重建的最小状态”。如果把海量 prompt/tool 输出都内嵌，恢复会变慢、schema 会变脆弱。

Failure Modes：常见低分回答

- 把全部上下文序列化成唯一真相
- 不版本化 schema

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q46 Agent Crash 后怎样 Resume，而不是重新开始?

难度：★★★★★ 高频：★★★★★ 岗位：Agent / Harness

一句话结论 load checkpoint -> reconcile in-flight side effects -> verify environment -> continue；关键是幂等与状态对账。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：load checkpoint -> reconcile in-flight side effects -> verify environment -> continue；关键是幂等与状态对账。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- pending action ledger、idempotency key、lease、recovery state machine。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
resume(run_id):
  cp = load_latest_checkpoint(run_id)
  reconcile(cp.pending_actions)
  verify_environment(cp.environment_version)
  acquire_lease(run_id)
  continue_from(cp.event_offset)
```

工程证据 / 研究结论

Evidence Crash recovery 的 hardest case 是：副作用已成功但 checkpoint 尚未持久化。只有 operation ledger + idempotency + reconciliation 才能可靠判断是否重做。

Failure Modes：常见低分回答

- 直接 replay 最后一条 tool call

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果工具已经产生副作用但响应丢失，系统如何确定下一步?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q47 如何支持 Human Interrupt?

难度： ★★★★☆ **高频：** ★★★★☆ **岗位：** Agent / Harness

一句话结论 暂停必须持久化状态并允许外部修改，再在同一 task identity 下恢复。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：暂停必须持久化状态并允许外部修改，再在同一 task identity 下恢复。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- interrupt point、checkpoint、approval payload、resume token、audit。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 阻塞进程等待 stdin
- 服务重启后丢失

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q48 Long-running Agent 如何避免 Plan Drift?

难度：★★★★☆ 高频：★★★☆☆ 岗位：Agent / Harness

一句话结论 把 immutable goal 与 mutable plan 分离，每个 sprint 后对环境事实和目标重新 reconcile。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：把 immutable goal 与 mutable plan 分离，每个 sprint 后对环境事实和目标重新 reconcile。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- goal anchor、progress review、replan trigger、constraint check。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把初始 plan 当圣经

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q49 如何判断 Coding Agent 真的完成任务?

难度：★★★★☆ 高频：★★★★★ 岗位：Agent / Harness

一句话结论 环境验证优先于模型自述：tests、E2E、acceptance criteria、diff review、security checks。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：环境验证优先于模型自述：tests、E2E、acceptance criteria、diff review、security checks。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 独立 evaluator/CI 运行，结果以 artifact 和 test report 为证据。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- “Done” 即完成
- 只跑单元测试不看真实行为

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q50 设计一个运行 12 小时的 Agent，要新增哪些机制?

难度：★★★★★ 高频：★★★★☆ 岗位：Agent / Harness

一句话结论 durable state、worker lease、checkpoint、context reset、budget、heartbeats、recovery、HITL、version pinning。

面试官在考什么

你是否具备 durable workflow 思维：进程、上下文和 worker 都可以消失，但任务不能丢。

Know-Why：为什么

长任务的关键是“外部化进度”。当上下文、进程或 worker 可以随时消失时，系统仍必须知道目标、已完成内容和下一步。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：durable state、worker lease、checkpoint、context reset、budget、heartbeats、recovery、HITL、version pinning。这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 把 agent worker 无状态化，状态外置
- 支持 task ownership 转移

Failure Modes：常见低分回答

- 一个进程从头跑到尾
- 没有环境版本

Trade-off：主动说出代价

频繁 checkpoint/reset 更可靠但增加 IO 与上下文重建成本；间隔太长则扩大故障重做范围。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第五章复盘：把 10 道题串成一个系统

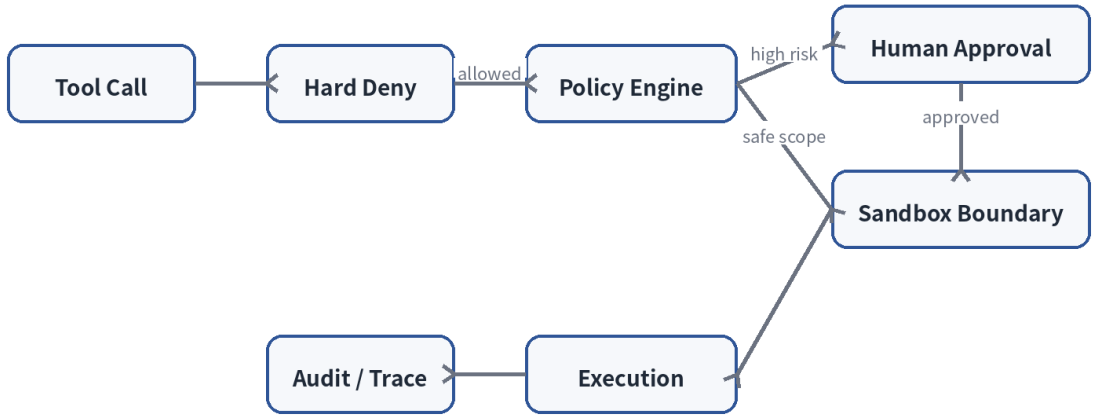
本章一句话 长任务依赖 initializer、sprint contract、incremental progress、environment verification、checkpoint 与 structured handoff，最终实现可恢复自治。

第六章 Permission / Sandbox / Security

本章核心 安全的核心是缩小 blast radius，并让授权、身份、凭证和执行边界成为系统原语。

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

图 4 Permission + Sandbox：安全不是“多问一次”，而是缩小可造成的伤害范围



本章面试目标

- 能从 blast radius 视角设计权限。
- 理解 sandbox 与 approval 的互补。
- 能设计 secret/identity/audit。

Q51 设计 Agent Permission System。

难度：★★★★☆ 高频：★★★★★ 岗位：Harness / Infra / Backend

一句话结论 至少有 hard deny、policy engine、human approval 三层，并结合 sandbox 缩小默认权限。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：至少有 hard deny、policy engine、human approval 三层，并结合 sandbox 缩小默认权限。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- risk classify -> policy -> approval -> execute -> audit。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
Tool Call
  -> Hard Deny
  -> Contextual Policy
  -> Risk Score
  -> [ALLOW | SANDBOX | ASK | DENY]
  -> Execute
  -> Audit + State Diff
```

工程证据 / 研究结论

Evidence Anthropic 的 Claude Code 安全实践把 permission prompts 与 sandbox 结合；其报告称 sandboxing 在内部使用中显著减少审批次数，同时缩小可访问文件和网络范围。[W7]

Failure Modes：常见低分回答

- 所有风险都靠弹窗
- 或全凭 prompt 自律

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q52 Permission Prompt 和 Sandbox 有什么区别?

难度：★★★★☆　　高频：★★★★★　　岗位：Harness / Infra / Backend

一句话结论 Permission 是授权决策；Sandbox 是技术隔离。安全边界应优先由执行环境强制，而非依赖注意力。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Permission 是授权决策；Sandbox 是技术隔离。安全边界应优先由执行环境强制，而非依赖注意力。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- workspace FS、network egress、process limits、capabilities
- 越界再审批

Failure Modes：常见低分回答

- “用户点了允许”就给全系统权限

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q53 什么是 Approval Fatigue?

难度：★★★★☆　　高频：★★★★☆　　岗位：Harness / Infra / Backend

一句话结论 高频确认会让用户机械通过，从而降低真实安全性；应让低风险动作在安全边界内自动执行。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：高频确认会让用户机械通过，从而降低真实安全性；应让低风险动作在安全边界内自动执行。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- risk-based approval、batch approval、sandbox、remember scoped grants。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Claude Code 团队明确讨论 approval fatigue：高通过率和高频弹窗会降低人的注意力，因此安全设计应减少无意义确认，而不是增加确认。[W7]

Failure Modes：常见低分回答

- 每个 read/write/bash 都弹窗

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q54 Tool Risk 如何分级?

难度：★★★★☆ 高频：★★★☆☆ 岗位：Harness / Infra / Backend

一句话结论 可用 Impact × Irreversibility × Scope × Sensitivity × External Side Effect 评估风险。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：可用 Impact × Irreversibility × Scope × Sensitivity × External Side Effect 评估风险。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 映射到 auto/sandbox/ask/dual approval/deny 五档。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只按工具名称分风险，不看参数和对象

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q55 Prompt Injection 为什么对 Agent 比 Chatbot 更危险?

难度：★★★★★ 高频：★★★★★ 岗位：Harness / Infra / Backend

一句话结论 不可信内容可能从“影响回答”升级为“触发真实动作”，因此数据与控制必须分离。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：不可信内容可能从“影响回答”升级为“触发真实动作”，因此数据与控制必须分离。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- taint untrusted content、policy at execution、secret isolation、allowlist network。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 网页里的文字能直接改变系统权限

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q56 Secret 应该怎么提供给 Agent?

难度：★★★★☆　　高频：★★★★☆　　岗位：Harness / Infra / Backend

一句话结论 尽量通过 credential broker 在执行层注入 scoped/short-lived token，而不是把 secret 暴露给模型。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：尽量通过 credential broker 在执行层注入 scoped/short-lived token，而不是把 secret 暴露给模型。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- secretless model context、proxy signing、scope、TTL、rotation。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- API key 放 system prompt 或日志

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q57 为什么 Sandbox 必须同时考虑 Filesystem 和 Network?

难度：★★★★★ 高频：★★★★★ 岗位：Harness / Infra / Backend

一句话结论 缺一会形成逃逸链：能读不能出网仍可改系统；能出网且能读敏感文件则可外泄。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：缺一会形成逃逸链：能读不能出网仍可改系统；能出网且能读敏感文件则可外泄。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- FS allowlist + egress proxy + subprocess inheritance + syscall/container boundary。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 的 sandbox 设计同时使用 filesystem isolation 与 network isolation，并指出单独使用其中一个都会留下明显逃逸/外泄路径。[W7]

Failure Modes：常见低分回答

- 只限制 cwd，但 shell 子进程能访问网络和 home

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q58 Least Privilege 在 Agent 中怎样实现?

难度：★★★★☆ 高频：★★★★☆ 岗位：Harness / Infra / Backend

一句话结论 身份与权限必须绑定用户、任务、资源、动作和时间，而不是给通用管理员凭证。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：身份与权限必须绑定用户、任务、资源、动作和时间，而不是给通用管理员凭证。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- scoped credential、branch/repo restriction、TTL、capability token。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 共享长期 admin token

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q59 哪些动作应该 Human-in-the-loop?

难度：★★★★☆　　高频：★★★★☆　　岗位：Harness / Infra / Backend

一句话结论 高影响、不可逆、涉及资金/外部沟通/敏感数据/权限提升的动作通常需人工确认。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：高影响、不可逆、涉及资金/外部沟通/敏感数据/权限提升的动作通常需人工确认。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 结合 expected harm、confidence、reversibility 动态决策。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把“发送邮件”一概允许或一概禁止

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?

- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q60 Agent Audit Log 应记录什么？

难度：★★★★☆　　高频：★★★☆☆　　岗位：Harness / Infra / Backend

一句话结论 记录谁以何目标、在什么上下文和授权下调用了什么工具、造成什么状态变化。

面试官在考什么

你是否能在 agent autonomy 与安全之间用系统机制做权衡，而不是只靠人工确认。

Know-Why：为什么

安全的核心不是相信模型，而是把可造成的伤害范围工程化地限制在最小边界，并对越界动作做显式授权。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：记录谁以何目标、在什么上下文和授权下调用了什么工具、造成什么状态变化。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- run/actor/model/tool/args/decision/result/timestamps/state diff/artifact refs。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只记录 final answer
- 日志包含明文 secret

Trade-off：主动说出代价

严格隔离提高安全但可能降低可用性；权限放宽提高自治，却必须用 scope、审计和可撤销性控制风险。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果 prompt injection 已经成功，什么系统边界仍能阻止真实伤害？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第六章复盘：把 10 道题串成一个系统

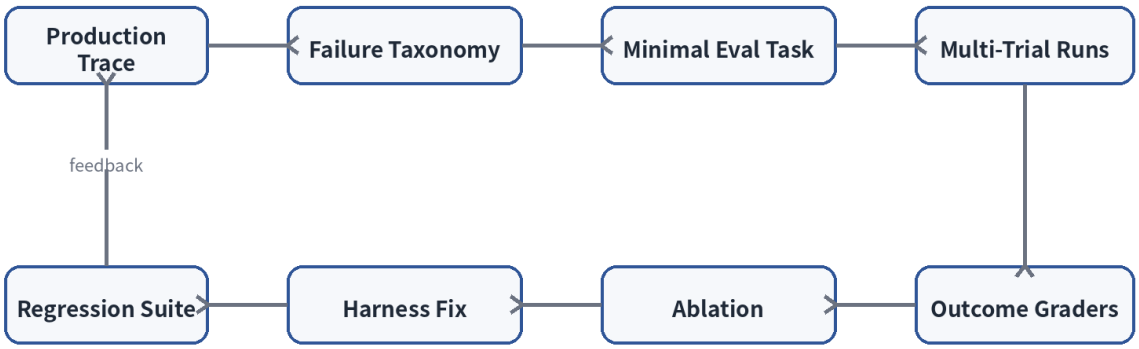
本章一句话 权限不是弹窗，而是 hard deny + contextual policy + sandbox + scoped credentials + audit 共同构成的 capability containment。

第七章 Evaluation / Observability

本章核心 Agent 评测必须面向 trajectory + outcome；生产 trace 要进入 eval 和 regression 闭环。

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

图 5 Agent Eval：从生产失败反推可重复的回归测试



本章面试目标

- 能用 task/trial/trace/outcome/grader 语言讨论 eval。
- 理解多 trial、pass@k/pass^k、judge calibration。
- 能把生产失败沉淀为 regression。

Q61 Agent Harness 和 Eval Harness 有什么区别?

难度：★★★★★ 高频：★★★★★ 岗位：Harness / Infra / Backend

一句话结论 Agent Harness 运行一次 Agent；Eval Harness 运行大量 task/trial 并评分、聚合和对比。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Agent Harness 运行一次 Agent；Eval Harness 运行大量 task/trial 并评分、聚合和对比。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- evaluation dataset + runner + grader + metrics + artifact store。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 的 Agent Eval 方法把 task、trial、trace、outcome、grader 分开建模，强调 Agent 评测对象实际上是 model × harness 的整体行为。[W7]

Failure Modes：常见低分回答

- 把线上 trace viewer 当完整评测系统

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q62 Task、Trial、Trace、Outcome、Grader 分别是什么?

难度：★★★★☆　　高频：★★★★☆　　岗位：Harness / Infra / Backend

一句话结论 Task 是问题，Trial 是一次随机执行，Trace 是轨迹，Outcome 是最终环境状态，Grader 判定成功。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Task 是问题，Trial 是一次随机执行，Trace 是轨迹，Outcome 是最终环境状态，Grader 判定成功。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 明确各层 ID，允许同 task 多 trial 和多 grader。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只保存最终文本，无法重放和归因

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q63 为什么应该优先 Eval Outcome，而不是 Tool Trajectory?

难度：★★★★★ 高频：★★★★★ 岗位：Harness / Infra / Backend

一句话结论 同一目标可能有多条合法路径；过度约束轨迹会奖励“像参考答案”而非真正成功。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：同一目标可能有多条合法路径；过度约束轨迹会奖励“像参考答案”而非真正成功。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 用 state/test/artifact grade 结果，只在安全或流程硬约束时检查 trajectory。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence 对于 Agent，合法成功路径往往不唯一，因此 outcome-based grading 通常比“必须按参考工具顺序执行”更稳健。轨迹检查应保留给安全/合规等硬约束。[W7]

Failure Modes：常见低分回答

- 规定必须按固定工具顺序

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q64 Code Grader、LLM Grader、Human Grader 怎么选?

难度：★★★★☆ 高频：★★★★☆ 岗位：Harness / Infra / Backend

一句话结论 能确定性判断就 code；开放质量用 LLM；用 human 建 gold set 和校准边界。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：能确定性判断就 code；开放质量用 LLM；用 human 建 gold set 和校准边界。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- hybrid grader、rubric、blind review、disagreement audit。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 所有题都用单一 LLM judge

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q65 pass@k 和 pass^k 区别是什么?

难度：★★★★★ 高频：★★★★★ 岗位：Harness / Infra / Backend

一句话结论 pass@k 衡量多次尝试至少一次成功的能力；pass^k 衡量每次都成功的可靠性。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：pass@k 衡量多次尝试至少一次成功的能力；pass^k 衡量每次都成功的可靠性。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 根据产品重试语义选择指标，并报告 trial count 与 confidence interval。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
pass@k = P(at least one success in k trials)
pass^k = P(all k trials succeed)

If single-trial success p = 0.75:
pass^3 = 0.75^3 = 42.2%
```

工程证据 / 研究结论

Evidence pass@k 更像“多试几次能不能找到一个解”，pass^k 更像“产品连续使用是否可靠”。面向最终用户的高价值 Agent 往往更关心后者。[W7]

Failure Modes：常见低分回答

- 只报 best-of-N 掩盖不稳定

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q66 Agent Eval 为什么一定要多 Trial?

难度：★★★★☆　　高频：★★★☆☆　　岗位：Harness / Infra / Backend

一句话结论 Agent 行为具有随机性和路径分叉，单次结果不足以估计真实成功率和稳定性。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Agent 行为具有随机性和路径分叉，单次结果不足以估计真实成功率和稳定性。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 固定 task set、多 seed、多模型温度或 provider noise，计算方差。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 一次 pass 就宣布提升

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q67 Capability Eval 和 Regression Eval 有什么不同?

难度：★★★★☆ **高频：**★★★★☆ **岗位：**Harness / Infra / Backend

> **一句话结论** Capability 留 headroom 找能力上限；Regression 接近 100% 防止已解决问题回退。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Capability 留 headroom 找能力上限；Regression 接近 100% 防止已解决问题回退。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 成熟 capability case 迁入 regression suite。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 所有题难度混在一个总分

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q68 怎样校准 LLM-as-Judge?

难度：★★★★★ 高频：★★★★☆ 岗位：Harness / Infra / Backend

一句话结论 用 human gold set 测 agreement，分析分歧，逐维 rubric，避免自评偏置。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：用 human gold set 测 agreement，分析分歧，逐维 rubric，避免自评偏置。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- pairwise、multi-judge、position swap、confidence threshold。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- judge prompt 从不版本化

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q69 Agent 会不会 Hack Eval?

难度：★★★★★ 高频：★★★☆☆ 岗位：Harness / Infra / Backend

一句话结论 会通过改测试、读答案、利用 grader 漏洞或改变环境来拿分，因此 eval 是对抗系统。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：会通过改测试、读答案、利用 grader 漏洞或改变环境来拿分，因此 eval 是对抗系统。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- isolate secrets、readonly tests、post-state check、tamper detection。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把 answer key 放 workspace

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 你的 grader 会不会被 Agent 利用？怎样验证?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q70 Production Trace 应如何变成 Eval Dataset?

难度：★★★★★ 高频：★★★★★ 岗位：Harness / Infra / Backend

一句话结论 生产失败 -> 根因分类 -> 最小复现 -> grader -> fix -> regression，是最重要的持续改进闭环。

面试官在考什么

你是否能用可重复实验区分“感觉更好”和“系统真的更好”。

Know-Why：为什么

没有 eval 的 Agent 优化不可重复。Agent 的随机轨迹决定了评测必须同时记录过程、结果和多次试验的稳定性。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：生产失败 -> 根因分类 -> 最小复现 -> grader -> fix -> regression，是最重要的持续改进闭环。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- sample/cluster traces，优先高频高损失 failure。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence 最有价值的 eval 数据往往来自生产 failure trace。将高损失失败压缩成最小复现并进入 regression，能让 Harness 改进形成复利。

Failure Modes：常见低分回答

- 只收集“有趣案例”，无回归

Trade-off：主动说出代价

更严格 eval 增加运行成本，但没有可靠 grader 的“优化”很容易过拟合几个样例。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 你的 grader 会不会被 Agent 利用？怎样验证？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第七章复盘：把 10 道题串成一个系统

本章一句话 把 Agent 行为拆成 Task/Trial/Trace/Outcome/Grader，并让生产失败持续进入 regression，才能建立真正的 eval-driven development。

第八章 Reliability / Distributed Agent Systems

本章核心 Agent Harness 本质上会遭遇分布式系统问题：重试、幂等、取消、并发、一致性和故障恢复。

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

本章面试目标

- 能把 retry/idempotency/cancellation/concurrency 讲到分布式系统级别。
- 能处理 partial failure 与 shared state。
- 能设计 durable worker。

Q71 Run、Thread、Turn 应该怎么建模?

难度：★★★★☆ 高频：★★★★☆ 岗位：Harness / Infra / Backend

一句话结论 Thread 表示长期会话/任务，Run 表示一次执行，Turn 表示一次 model-action-observation 循环。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Thread 表示长期会话/任务，Run 表示一次执行，Turn 表示一次 model-action-observation 循环。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 独立 ID、parent-child、event offset、status machine。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把 HTTP request 当 run 的唯一生命周期

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q72 Tool Retry 应该怎么设计?

难度：★★★★☆　　高频：★★★☆☆　　岗位：Harness / Infra / Backend

一句话结论 按 transient/permanent/semantic error 分类，重试预算和模型纠参路径应分开。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：按 transient/permanent/semantic error 分类，重试预算和模型纠参路径应分开。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- backoff+jitter、retry-after、circuit breaker、semantic repair。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 所有错误都指数重试

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q73 Agent Tool 能实现 Exactly Once 吗?

难度：★★★★★　　高频：★★★★★　　岗位：Harness / Infra / Backend

一句话结论 严格 exactly-once 很难；工程上通常用 at-least-once + idempotency + dedupe + reconciliation。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：严格 exactly-once 很难；工程上通常用 at-least-once + idempotency + dedupe + reconciliation。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- persist operation status before/after side effect，支持查询结果。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
Delivery semantics: at-least-once
Correctness strategy:
  idempotency key
+ persisted operation status
+ deduplication
+ reconciliation query
+ compensating action
```

Failure Modes：常见低分回答

- 相信网络请求返回一次就执行一次

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果工具已经产生副作用但响应丢失，系统如何确定下一步?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q74 Cancellation 怎么设计?

难度： ★★★★☆　　**高频：** ★★★★☆　　**岗位：** Harness / Infra / Backend

> **一句话结论** 取消要沿 call tree 传播到 LLM、tool、subagent、sandbox process，并保存可恢复状态。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：取消要沿 call tree 传播到 LLM、tool、subagent、sandbox process，并保存可恢复状态。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- cancellation token、cooperative cancel、force kill、cleanup、checkpoint。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只停止前端 streaming

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q75 Multi-Agent 并发怎么控制?

难度：★★★★☆　　高频：★★★☆☆　　岗位：Harness / Infra / Backend

一句话结论 需要 global/per-user/per-run/per-tool/provider 多级并发和速率控制。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：需要 global/per-user/per-run/per-tool/provider 多级并发和速率控制。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- queue、semaphore、token bucket、priority、backpressure。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 每个 agent 自己 spawn 无全局配额

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q76 Shared Agent State 怎么保证一致性?

难度：★★★★★ 高频：★★★★☆ 岗位：Harness / Infra / Backend

一句话结论 会出现 lost update、stale read、duplicate action，需要版本、事务或单写者模型。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：会出现 lost update、stale read、duplicate action，需要版本、事务或单写者模型。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- optimistic locking、compare-and-swap、event sourcing、transaction。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 多个 worker 直接改同一个 JSON 文件

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q77 Tool Partial Failure 怎么处理?

难度：★★★★★ 高频：★★★★☆ 岗位：Harness / Infra / Backend

一句话结论 返回结构化部分成功状态，让 Agent 做补偿，而不是把复合动作压成一个 bool。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：返回结构化部分成功状态，让 Agent 做补偿，而不是把复合动作压成一个 bool。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- saga/compensation、step status、operation id。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 第一步成功第二步失败却整体 retry

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果工具已经产生副作用但响应丢失，系统如何确定下一步?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q78 如何设计 Agent Rate Limiting?

难度： ★★★★☆ **高频：** ★★★☆☆ **岗位：** Harness / Infra / Backend

> **一句话结论** Agent 请求会内部放大，必须限制 run、turn、tokens、tools、subagents、cost，而非只限 HTTP QPS。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Agent 请求会内部放大，必须限制 run、turn、tokens、tools、subagents、cost，而非只限 HTTP QPS。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- hierarchical budgets + admission control + per-provider limits。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 一个用户请求触发数百模型调用

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q79 Durable Agent Worker 怎么设计？

难度：★★★★★ 高频：★★★★★ 岗位：Harness / Infra / Backend

> **一句话结论** worker 负责计算，不拥有唯一状态；任务状态应在 queue/checkpoint/event store 中可转移。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：worker 负责计算，不拥有唯一状态；任务状态应在 queue/checkpoint/event store 中可转移。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- lease/heartbeat、visibility timeout、idempotent resume、dead-letter queue。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
API -> Task Queue -> Worker Lease
                  |
                  +-> Agent Runtime
                  +-> Checkpoint Store
                  +-> Event Log
                  +-> Artifact Store

Worker can die; task state survives.
```

Failure Modes：常见低分回答

- worker crash = task 丢失

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据？
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好？
- 如果要求成本下降 40%，你会优先移除或简化哪一层？

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q80 什么叫 Brain 与 Hands 解耦?

难度：★★★★★ 高频：★★★★☆ 岗位：Harness / Infra / Backend

一句话结论 把 reasoning/orchestration 与执行环境解耦，使 Harness 可独立演进、按需创建 sandbox，并支持多种执行端。

面试官在考什么

你是否能把 Agent 当真实生产服务，处理重复、并发和部分失败。

Know-Why：为什么

一旦 Agent 能执行真实副作用，它就继承了分布式系统的经典难题：重复、丢失、乱序、并发、部分失败和恢复。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：把 reasoning/orchestration 与执行环境解耦，使 Harness 可独立演进、按需创建 sandbox，并支持多种执行端。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- stable execute interface、remote sandbox、credential proxy、artifact channel。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 2026 Managed Agents 提出 brain/hands 解耦：让 reasoning harness 与 execution environments 通过稳定接口连接，以便二者独立扩展和演进。[W9]

Failure Modes：常见低分回答

- 模型进程和执行机紧耦合

Trade-off：主动说出代价

强一致和复杂事务提高正确性，却增加延迟和实现复杂度；需要按副作用风险选择语义。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第八章复盘：把 10 道题串成一个系统

本章一句话 工具副作用使 Agent 进入 distributed systems 世界；幂等、lease、concurrency control、partial

failure 和 event log 是可靠性的基本功。

第九章 Cost / Latency / Performance

本章核心 优化目标不是“最强模型”，而是 Quality × Reliability × Latency × Cost 的 Pareto 前沿。

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

本章面试目标

- 能拆解 agent latency critical path。
- 会做 model routing、cache、parallelism、budget。
- 能用 Pareto 而不是单指标判断优化。

Q81 Agent Latency 怎么拆?

难度：★★★☆☆ 高频：★★★★★ 岗位：Harness / AI Systems

一句话结论 总延迟由 TTFT、generation、serial turns、tool、network、sandbox、queue、retry 共同组成。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：总延迟由 TTFT、generation、serial turns、tool、network、sandbox、queue、retry 共同组成。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- trace critical path，优先减少串行 round-trip 而非只优化单次模型。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只盯模型 tokens/sec

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q82 怎样做 Model Routing?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Harness / AI Systems

一句话结论 按任务难度、风险、工具复杂度和质量要求选模型，目标是 quality per dollar/second。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：按任务难度、风险、工具复杂度和质量要求选模型，目标是 quality per dollar/second。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- small model routing/summarization，frontier model planning/high-risk reasoning。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 所有步骤都用最强模型

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q83 Parallel Agent 为什么可能更快但更贵?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Harness / AI Systems

一句话结论 并行减少 wall-clock critical path，但通常不减少总 tokens，且可能增加重复探索。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：并行减少 wall-clock critical path，但通常不减少总 tokens，且可能增加重复探索。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 只并行独立高价值分支，设置 speculative cancellation。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把所有候选路径全并行

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q84 Prompt Caching 在 Agent 中为什么特别重要?

难度：★★★☆☆ 高频：★★★☆☆ 岗位：Harness / AI Systems

一句话结论 Agent 每 turn 重复 system/tool schemas/static context，稳定前缀可显著降低输入成本和延迟。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Agent 每 turn 重复 system/tool schemas/static context，稳定前缀可显著降低输入成本和延迟。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 按 static->semi-static->dynamic 排列，版本化前缀。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 频繁改变前缀导致缓存失效

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?

- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q85 Tool Result 为什么必须 Token Efficient?

难度： ★★★☆☆ **高频：** ★★★★☆ **岗位：** Harness / AI Systems

> **一句话结论** 工具输出也是 context 成本；应返回决策所需最小信息和 artifact reference。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：工具输出也是 context 成本；应返回决策所需最小信息和 artifact reference。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- filter、paginate、aggregate、top-k、cursor、artifact_id。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把 50MB 日志直接回模型

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q86 Streaming 对 Agent 有什么价值?

难度： ★★★☆☆ **高频：** ★★★★☆ **岗位：** Harness / AI Systems

> **一句话结论** 除 UX 外，还支持进度、取消和下游流水线；但必须区分 speculative text 与 committed action。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：除 UX 外，还支持进度、取消和下游流水线；但必须区分 speculative text 与 committed action。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- event stream 分类型：thought-free status、tool event、artifact、final。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 把中间文本当已执行事实

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q87 模型升级后 Harness 一定应该保持不变吗?

难度：★★★★☆　　高频：★★★★★　　岗位：Harness / AI Systems

一句话结论 Harness 常编码模型弱点假设；模型增强后旧补丁可能变成负担，应重新做 ablation。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Harness 常编码模型弱点假设；模型增强后旧补丁可能变成负担，应重新做 ablation。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- model rollout 前跑 harness ablation matrix 和 regression。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Managed Agents 也强调 Harness 会编码“模型做不到什么”的假设；模型进步后旧机制可能过时，因此 model rollout 应重新消融 Harness。[W9]

Failure Modes：常见低分回答

- 只替换 model name

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q88 Sandbox Cold Start 怎么优化?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Harness / AI Systems

> **一句话结论** lazy provisioning、warm pool、snapshot、image/repo cache，以及 brain-first/hands-on-demand。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：lazy provisioning、warm pool、snapshot、image/repo cache，以及 brain-first/hands-on-demand。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 按任务预测预热
- 安全隔离不能因性能优化被绕过

Failure Modes：常见低分回答

- 长期复用脏 sandbox

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q89 怎样给 Agent 设置 Cost Budget?

难度：★★★☆☆ 高频：★★★★☆ 岗位：Harness / AI Systems

一句话结论 预算应分层并可观察，让 Agent/Orchestrator 在剩余 token、时间、工具次数下改变策略。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：预算应分层并可观察，让 Agent/Orchestrator 在剩余 token、时间、工具次数下改变策略。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- run budget + per-tool budget + subagent allocator + hard stop。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 事后统计成本但运行时无控制

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q90 如何判断 Agent Optimization 是否真的有效?

难度：★★★★☆ 高频：★★★★★ 岗位：Harness / AI Systems

一句话结论 同时报告 solve rate、reliability、latency、tokens、cost、turns、tool calls、human intervention 和 failure severity。

面试官在考什么

你是否能把性能问题拆成关键路径，并做多目标优化。

Know-Why：为什么

Agent 性能优化是多目标优化。最强模型不一定是最佳系统，减少串行回合和无效上下文通常比单纯换模型更划算。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：同时报告 solve rate、reliability、latency、tokens、cost、turns、tool calls、human intervention 和 failure severity。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 比较 Pareto frontier，不只看单指标涨幅。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 成功率 +2% 但成本翻 5 倍仍称优化

Trade-off：主动说出代价

降低 latency 往往通过并行、缓存和更小模型实现，但可能提高成本、陈旧风险或降低边缘任务质量。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 如果要求成本下降 40%，你会优先移除或简化哪一层?

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第九章复盘：把 10 道题串成一个系统

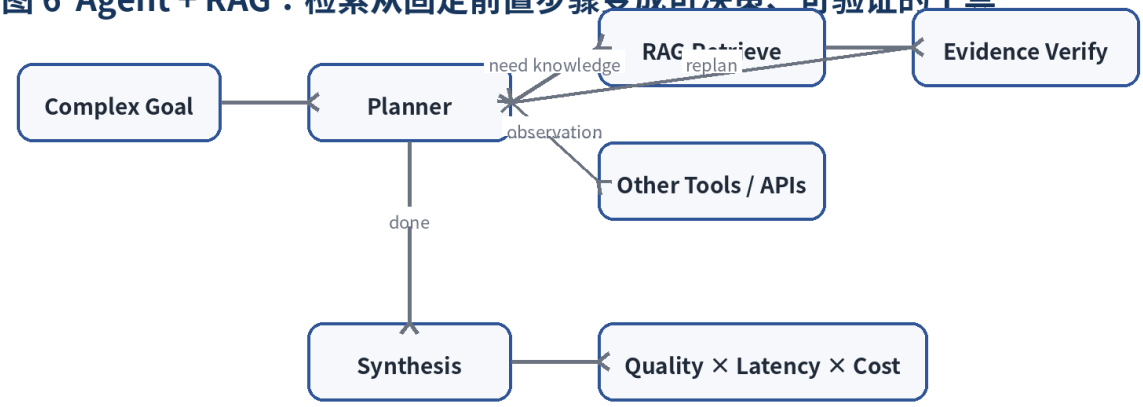
本章一句话 性能优化先看 critical path，再看 tokens/cost/quality；缓存、并行、模型路由和预算控制都必须通过 Eval 比较 Pareto。

第十章 System Design / Coding / Debugging

本章核心 把前九章组合成完整系统设计：Coding Agent、Research Agent、平台、多租户、Trace、Checkpoint 与回归定位。

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

图 6 Agent + RAG：检索从固定前置步骤变成可决策、可验证的工具



本章面试目标

- 能画完整 Coding/Research/Support/Platform 架构。
- 能写 checkpoint/trace/permission schema。
- 能从 solve-rate regression 做证据驱动调试。

Q91 系统设计：设计一个 Coding Agent。

难度：★★★★★ 高频：★★★★★ 岗位：Senior / Staff Agent Systems

一句话结论 核心是 repo context、edit/search/test tools、sandbox、permission、checkpoint、git、trace 和环境验证闭环。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：核心是 repo context、edit/search/test tools、sandbox、permission、checkpoint、git、trace 和环境验证闭环。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- understand -> inspect -> plan -> edit -> test -> debug -> verify
- 长任务按 sprint 切分

```
User -> Run Manager -> Context Builder -> LLM
                         -> Policy -> Tool Router
                                   -> Search/Read/Edit/Test
                                   -> Sandboxed Repo
                         -> Checkpoint / Trace / Eval
```

工程证据 / 研究结论

Evidence OpenAI Codex/API/Cloud Agents 公开岗位将 execution loop、sandbox、state/memory、orchestration、eval、observability、tokens/latency/cost 明确列为核心工作，Coding Agent 系统设计应覆盖这些层。[W1][W2][W3]

Failure Modes：常见低分回答

- 只描述“模型调用 shell”

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q92 系统设计：设计一个 Deep Research Agent。

难度：★★★★☆　　高频：★★★★☆　　岗位：Senior / Staff Agent Systems

一句话结论 Lead Agent 做规划与综合，subagents 并行搜索；证据、来源质量、去重、引用验证和搜索预算是核心。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Lead Agent 做规划与综合，subagents 并行搜索；证据、来源质量、去重、引用验证和搜索预算是核心。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- plan -> fan-out -> evidence store -> synthesis -> citation verify。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

工程证据 / 研究结论

Evidence Anthropic 多 Agent Research 系统把 lead agent、subagents、parallel search、evidence synthesis 组合起来；其工程价值来自 breadth 与上下文隔离，而不是简单增加 Agent 数量。[W9]

Failure Modes：常见低分回答

- 多 Agent 搜索无证据归并和来源质量控制

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q93 系统设计：设计 Customer Support Agent。

难度：★★★★☆　　高频：★★★☆☆　　岗位：Senior / Staff Agent Systems

一句话结论 身份、政策检索、客户状态、动作权限、升级人工和审计比聊天体验更重要。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：身份、政策检索、客户状态、动作权限、升级人工和审计比聊天体验更重要。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- verify identity -> retrieve policy -> reason -> action -> verify state -> confirm。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 能回答退款政策就自动退款

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q94 系统设计：设计 Multi-Tenant Agent Platform。

难度：★★★★★ 高频：★★★★☆ 岗位：Senior / Staff Agent Systems

一句话结论 必须隔离 tenant/user/credential/sandbox/memory/cache/log，并统一 run、policy、budget、observability 抽象。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：必须隔离 tenant/user/credential/sandbox/memory/cache/log，并统一 run、policy、budget、observability 抽象。这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- tenant-aware authz、resource quota、per-tenant encryption、data lineage。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

Failure Modes：常见低分回答

- 只在 prompt 里写 tenant_id

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q95 Coding：实现 Production Agent Loop，你会加哪些机制?

难度：★★★★★ 高频：★★★★★ 岗位：Senior / Staff Agent Systems

一句话结论 生产 loop 需要 state load、context build、model call、policy、tool execution、trace、checkpoint、budget、interrupt。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：生产 loop 需要 state load、context build、model call、policy、tool execution、trace、checkpoint、budget、interrupt。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 把每一步做成可重放 event，并处理 timeout/retry/cancel。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
while state.running:
    state = store.load(run_id)
    ctx = context_builder.build(state)
    response = llm(ctx)
    for call in response.tool_calls:
        policy = permission_engine.evaluate(call, state)
        result = execute_with_retry(call, policy)
        event_log.append(result)
    store.checkpoint(state)
    enforce_budget(state)
```

Failure Modes：常见低分回答

- 同步单函数 while loop 无状态持久化

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q96 Coding：设计 Permission Engine。

难度：★★★★☆ 高频：★★★☆☆ 岗位：Senior / Staff Agent Systems

一句话结论 输入至少包含 actor、agent、tool、arguments、environment，输出 allow/deny/ask/sandbox + reason。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：输入至少包含 actor、agent、tool、arguments、environment，输出 allow/deny/ask/sandbox + reason。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- rule engine + risk score + policy version + audit + pre/post guard。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
Decision evaluate(
  actor, agent, tool, arguments, environment
) -> {ALLOW | DENY | ASK | SANDBOX, reason, scope}
```

Failure Modes：常见低分回答

- 只按 tool name allowlist

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q97 Coding：设计 Agent Checkpoint Schema。

难度： ★★★★★ **高频：** ★★★★☆ **岗位：** Senior / Staff Agent Systems

一句话结论 Schema 应保存恢复所需的最小不可重建状态并带版本号，artifact 用引用而非内嵌。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：Schema 应保存恢复所需的最小不可重建状态并带版本号，artifact 用引用而非内嵌。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- run/thread/version/status/goal/plan/state/event_offset/env/budget/pending_actions。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
{
  "run_id": "...", "version": 42,
  "status": "RUNNING",
  "goal_ref": "artifact://goal",
  "task_state": {...},
  "event_offset": 291,
  "environment": {"sandbox_id": "...", "git_commit": "..."},
  "budget": {"tokens": 12345, "cost": 2.14},
  "pending_actions": []
}
```

Failure Modes：常见低分回答

- checkpoint 过大、不可迁移、无 schema version

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q98 Coding：Agent Trace Schema 应怎么设计?

难度： ★★★★★ **高频：** ★★★★☆ **岗位：** Senior / Staff Agent Systems

> **一句话结论** 用 Run -> Turn -> Model/Tool/Handoff/Guardrail spans 表达因果，记录 latency/token/cost/error 和数据引用。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：用 Run -> Turn -> Model/Tool/Handoff/Guardrail spans 表达因果，记录 latency/token/cost/error 和数据引用。 这类控制点一

旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- trace_id/span_id/parent_id + typed attributes + sensitive-data policy。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
RunSpan
  TurnSpan
    ModelSpan
    ToolSpan
    ToolSpan
  TurnSpan
    ModelSpan
  HandoffSpan / GuardrailSpan / CustomSpan
```

Failure Modes：常见低分回答

- 日志无 parent-child，无法还原 critical path

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q99 Debug：Agent Solve Rate 从 72% 掉到 58%，怎么查?

难度：★★★★★　　高频：★★★★★　　岗位：Senior / Staff Agent Systems

> **一句话结论** 先排 eval noise，再做版本矩阵、failure clustering、trace diff 和单变量 ablation，最后形成 regression。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：先排 eval noise，再做版本矩阵、failure clustering、trace diff 和单变量 ablation，最后形成 regression。 这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- Model/Context/Tool/Policy/Env/Eval/Infra 七层定位。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
1. Reproduce + estimate eval variance
2. Freeze task set and compare versions
3. Cluster failed traces
4. Ablate one variable at a time
5. Verify root cause with minimal cases
6. Ship fix + regression suite
```

工程证据 / 研究结论

Evidence OpenAI Codex Agents 岗位明确要求能区分 harness issue、model behavior、inference/runtime issue 和 product failure，并通过 experiments/ablations 将 messy failures 变成 durable improvements。[W1]

Failure Modes：常见低分回答

- 直接改 prompt
- 只看成功案例

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

Q100 给你一个 Agent，你如何系统地提升 30% Performance?

难度：★★★★★ 高频：★★★★★ 岗位：Senior / Staff Agent Systems

一句话结论 定义目标 -> baseline -> trace -> failure taxonomy -> hypothesis -> ablation -> fix -> regression -> production validation。

面试官在考什么

你能否把前面所有原语组合成一套可落地、可调试、可演进的系统。

Know-Why：为什么

系统设计题考查的是把模型能力、后端可靠性、安全边界和评测闭环统一起来，而不是画一个“LLM + Tools”的方框。

针对本题，关键不是记住术语，而是识别系统中的“控制点”：定义目标 -> baseline -> trace -> failure taxonomy -> hypothesis -> ablation -> fix -> regression -> production validation。这类控制点一旦缺失，模型即使单次推理正确，也可能因为重试、上下文污染、环境状态、权限或错误反馈而在真实任务中失败。

Know-How：怎么做

- 优化工具、上下文、模型路由、planner、memory、evaluator、retry、budget，并比较 Pareto。
- 把这一机制做成可配置、可追踪、可评测的 runtime primitive。

```
Objective -> Baseline -> Trace -> Failure Taxonomy
          -> Hypothesis -> Ablation -> Fix
```

```
          -> Regression -> Production Validation

Always measure: Quality / Reliability / Latency / Cost / Safety
```

工程证据 / 研究结论

Evidence 真正的 30% 提升通常来自多个小杠杆叠加：tool、context、runtime、eval、model routing 和 recovery。关键是用 ablation 证明每个杠杆的因果贡献。

Failure Modes：常见低分回答

- 无 eval 的“凭感觉优化”
- 把性能等同模型分数

Trade-off：主动说出代价

系统设计越通用越容易过度抽象；面试中应先给针对场景的最小闭环，再说明扩展点。

面试追问

- 如果这个机制失败，你会在 Trace 中看到什么证据?
- 如何构造一个最小 Eval 来证明你的设计真的比 baseline 更好?
- 请把设计落到核心数据模型、关键 API 和两个 SLO 上。

Offer 级回答模板

建议用 4 段式回答：①先给结论和适用边界；②解释 failure mode，也就是“为什么需要它”；③给出最小实现和关键数据结构；④主动给出 trade-off + eval 指标。这样能把概念答案升级为工程答案。

第十章复盘：把 10 道题串成一个系统

本章一句话 系统设计题最终考的是闭环：Context -> Reason -> Policy -> Execute -> Verify -> Persist -> Observe -> Eval，而不是画框架 Logo。

附录 A 30 天 Agent Harness 面试冲刺计划

目标不是每天刷固定题数，而是让知识从“名词”升级到“可口述系统设计”。每 5 天完成一次闭环：学习 -> 口述 -> 白板设计 -> Eval/Debug -> 复盘。

阶段	主题	必须产出的 Artifact
Day 1-5	Loop / Tool / Context	手写最小 Agent Loop；实现 4 个工具；做一次 tool selection eval。
Day 6-10	Memory / RAG / Multi-Agent	设计 working/long-term memory；实现 agentic retrieval；比较 single vs multi-agent。
Day 11-15	Long-running / Checkpoint	实现 progress artifact、checkpoint、resume；模拟 crash 与 side-effect reconciliation。
Day 16-20	Security / Permission	设计 policy engine、sandbox threat model、scoped credentials 和 audit trace。
Day 21-25	Eval / Distributed Systems	构建 30 个 task eval set；多 trial；做 retry/idempotency/cancellation 设

		计题。
Day 26-30	System Design / Mock	每天 1 道 45 分钟系统设计；Q91/Q92/Q94/Q99/Q100 至少各复述两次。

每天 45 分钟训练法

13. 10 分钟：不看答案口述 3 题。
14. 15 分钟：选择 1 题画状态机/数据模型/调用链。
15. 10 分钟：为该设计列 5 个 failure modes。
16. 10 分钟：设计最小 eval 与两个可观测指标。

附录 B Agent System Design 白板模板

```
1. Clarify Goal / Success Metric
2. Environment & Trust Boundary
3. Observation / Context Sources
4. Action Space / Tool Registry
5. Agent Loop / Orchestration
6. State / Memory / Checkpoint
7. Permission / Identity / Sandbox
8. Failure Recovery / Idempotency
9. Eval / Trace / Monitoring
10. Cost / Latency / Capacity
11. Rollout / Regression / Ablation
```

面试技巧 先画“最小闭环”，再补横切能力。不要一上来画 20 个微服务。面试官通常更关心你是否明确成功条件、状态语义、失败恢复和安全边界。

建议主动给出的 8 个 SLO / 指标

- Task success / solve rate
- pass^k 或稳定成功率
- p50 / p95 end-to-end latency
- 平均 turns / tool calls
- token / cost per successful task
- human intervention rate
- unsafe/blocked action rate
- resume success / recovery time

附录 C Agent Failure Taxonomy（调试总表）

层	典型症状	应查看的证据	典型修复
Model	推理错误、误解目标、过早结束	model response / reasoning-independent signals	换模型或 targeted prompting
Context	遗漏关键信息、污染、过时	retrieval set / token composition	filter / compact / reset / retrieval
Tool Interface	选错工具、参数错误	tool-call distribution / schema errors	rename / schema / examples / retrieval
Tool Runtime	timeout、partial failure	tool latency/error / operation state	retry / idempotency / saga
Policy/Security	误拒绝、越权	policy decision trace	risk model / scoped grants / sandbox
State	丢进度、stale/冲突	checkpoint / version / event log	CAS / event sourcing / reconciliation
Environment	依赖漂移、脏状态	env hash / git / package state	pin / reset / verify baseline
Eval	grader 偏差、泄漏	human disagreement / tamper evidence	calibration / isolation / new grader
Infra	rate limit、容量、网络	provider metrics / queue / fleet	backoff / admission / capacity

调试原则

先判断“失败发生在哪一层”，再改那一层。Agent 系统最常见的无效工作，是把工具设计问题、环境问题、grader 问题全部用 prompt patch 掩盖。Prompt 是一个控制杠杆，但不是通用创可贴。

附录 D 参考资料与延伸阅读

所附资料

- [B1] Micheal Lanham, AI Agents in Action, Manning. 重点章节：Actions/Tools；Memory/Knowledge；Reasoning/Evaluation；Planning/Feedback。
- [B2] 凌峰，《AI Agent 开发与应用：基于大模型的智能体构建》。重点：智能体架构、上下文与记忆、RESTful API、向量数据库、ReAct、多智能体与案例实践。
- [B3] 《AI Agent 智能工作流》。用于补充 Agent/Workflow/Tool 调用和多 Agent 的应用视角。
- [B4] 《字节跳动 RAG 实践手册》。重点：检索触发与查询理解、索引/生成评估、全链路监控、成本、安全、Agent+RAG、故障复盘。

公开一手工程资料（检索日期：2026-09-02）

- [W1] OpenAI, AI Systems Engineer - Codex Agents. https://openai.com/careers/ai-systems-engineer-codex-agents-san-francisco/
- [W2] OpenAI, Software Engineer - API Agents. https://openai.com/careers/software-engineer-api-agents-san-francisco/
- [W3] OpenAI, Software Engineer - Cloud Agents / Codex Core Agents. https://openai.com/careers/software-engineer-cloud-agents-san-francisco/ ; https://openai.com/careers/software-engineer-codex-core-agents-san-francisco/
- [W4] Anthropic, Effective harnesses for long-running agents (2025-11-26). https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
- [W5] Anthropic, Harness design for long-running application development (2026-03-24). https://www.anthropic.com/engineering/harness-design-long-running-apps
- [W6] Anthropic, Effective context engineering for AI agents (2025-09-29). https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
- [W7] Anthropic, Demystifying evals for AI agents (2026-01-09); Beyond permission prompts: Claude Code sandboxing (2025-10-20). https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents ; https://www.anthropic.com/engineering/claude-code-sandboxing
- [W8] Yang et al., SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024. https://arxiv.org/abs/2405.15793
- [W9] Anthropic, Building Effective Agents (2024-12-19); Writing effective tools for agents (2025-09-11); Scaling Managed Agents (2026-04-08).
- [W10] OpenAI Agents SDK docs: Running agents / Handoffs / Agent orchestration / Tracing. https://openai.github.io/openai-agents-python/

如何继续扩展这 100 题

建议把真实面试或项目中出现的新失败按“Failure Taxonomy -> Minimal Eval -> Root Cause -> Harness Primitive”沉淀到题库。这样题库会从静态知识集合变成你自己的 Agent Engineering 操作系统。

最后的 20 题优先级 如果只有一周：优先 Q1、Q2、Q7、Q11、Q16、Q21、Q24、Q29、Q31、Q36、Q45、Q46、Q51、Q52、Q63、Q65、Q73、Q91、Q99、Q100。它们覆盖 Harness 的主干。